

*Universidad Rafael Landívar
Facultad de Ingeniería
Ingeniería Informática y sistemas
Programación avanzada*



GoXelaDelivery

Fase No.2

(Documento técnico)

Desarrolladores

*Edwin Stevan Cox Tay
Anllela Yasmin Perez Perez*

Septiembre 2026, Quetzaltenango

ÍNDICE

1. **Lógica del programa**
 - 1.1. Validación de datos
 - 1.2. Condiciones y flujo de procesos
 - 1.3. Ejemplo de validación en la creación de pedidos
2. **Programación Orientada a Objetos dentro del programa**
 - 2.1. Herencia jerárquica
 - 2.1.1. Herencia de Persona: Cliente y Repartidor
 - 2.1.2. Herencia de Vehículo: Bicicleta, Motocicleta y Automóvil
 - 2.2. Polimorfismo
 - 2.2.1. Sobreescritura de métodos en el registro de personas
 - 2.3. Encapsulamiento
 - 2.3.1. Declaración de atributos privados
 - 2.3.2. Uso de métodos Get y Set
 - 2.3.3. Validación de datos
3. **Dificultades durante el desarrollo**
 - 3.1. Dificultades durante la fase uno
 - 3.1.1. Definición de la lógica del programa
 - 3.1.2. Tipos de variables
 - 3.1.3. Relaciones entre clases
 - 3.1.4. Relación entre vehículos y repartidores
 - 3.2. Dificultades durante la fase de desarrollo
 - 3.2.1. Creación y manejo de repositorios
 - 3.2.2. Creación y manejo de ramas
 - 3.2.3. Conflictos durante los commits
 - 3.2.4. Resolución e integración de cambios
4. **Pruebas del programa**
(imágenes)
 - 4.1. Menú principal
 - 4.2. Menú de registro de vehículos
 - 4.3. Ingreso de datos y especificaciones
 - 4.4. Registro de paquetes
 - 4.5. Lista de vehículos registrados
 - 4.6. Reporte de capacidad
 - 4.7. Creación de entregas
 - 4.8. Registro de clientes
 - 4.9. Registro de paquetes
 - 4.10. Proceso de entrega
 - 4.11. Advertencias y validaciones del programa
5. **Observaciones y limitaciones del programa**

Este punto fue explicado gráficamente mediante material visual durante la fase uno. La lógica del programa se basa principalmente en la validación de datos y el cumplimiento de condiciones previamente establecidas para controlar el flujo de cada proceso.

La mayoría de las secciones requieren que el usuario ingrese la información de manera manual. A partir de estos datos, el sistema realiza diferentes validaciones antes de permitir que una determinada acción sea ejecutada.

Por ejemplo, para realizar un pedido, el sistema verifica previamente que el cliente se encuentre registrado. Si el cliente no existe en el sistema, la operación es bloqueada y no se permite continuar con el proceso. Si el cliente está registrado, el sistema permite continuar con el ingreso y procesamiento del pedido.

De esta manera, la lógica del programa establece reglas y condiciones que determinan qué acciones puede realizar el usuario y en qué momento, evitando procesos con información incompleta o que no cumplan con los requisitos establecidos.

Programación orientada a objetos dentro del programa

Herencia: Se implementó el concepto de herencia jerárquica dentro de la estructura del sistema, permitiendo organizar las clases de acuerdo con sus características y comportamientos comunes.

Un ejemplo es la clase Persona, de la cual heredan las clases Cliente y Repartidor. De esta manera, ambas clases pueden reutilizar los atributos y métodos definidos en la clase base, incorporando posteriormente sus propias características y funcionalidades específicas.

También se aplicó este concepto en la estructura de Vehículo, utilizando una clase base de la cual se derivan las clases Bicicleta, Motocicleta y Automóvil. Esto permite mantener una estructura organizada y reutilizar código, evitando la duplicación de atributos y métodos comunes.

La implementación de esta jerarquía facilita la reutilización de código, organización del sistema y aplicación de los principios de la programación orientada a objetos.

```
1  using System;
2
3  namespace GoXelaDelivery
4  {
5      public class Automovil : Vehiculo
6      {
7          public Automovil(string codigo, string placa, string marca, string modelo, double capacidadMaxima, double costoOperativo)
8              : base(codigo, placa, marca, modelo, capacidadMaxima, costoOperativo)
9          {
10          }
11
12          public override bool PuedeTransportar(Paquete paquete)
13          {
14              if (paquete == null) return false;
15              return paquete.Peso <= CapacidadMaxima;
16          }
17      }
18  }
19  }
```

polimorfismo

```
public override void MostrarInformacion()
{
    base.MostrarInformacion();
    Console.WriteLine($"Dirección: {Direccion}");
    Console.WriteLine($"Correo: {correo}");
}
```

```
public virtual void MostrarInformacion()
{
    Console.WriteLine($"Nombre: {Nombre}");
    Console.WriteLine($"Código: {Codigo}");
    Console.WriteLine($"Teléfono: {Telefono}");
}
```

El polimorfismo se implementó en el proceso de registro de personas. Dependiendo del tipo de persona que se registre, ya sea Cliente o Repartidor, el sistema utiliza la sobreescritura de métodos para ejecutar un comportamiento específico según la clase correspondiente.

Esto permite que un mismo método pueda tener diferentes implementaciones dependiendo del objeto que lo utilice. De esta manera, se facilita el proceso de registro y se mantiene una estructura más organizada y flexible, permitiendo que cada tipo de persona maneje sus propios datos y características de acuerdo con las necesidades del sistema.

Encapsulamiento

```
public abstract class Vehiculo
{
    private string codigo;
    private string placa;
    private string marca;
    private string modelo;
    private double capacidadMaxima;
    private double costoOperativo;
    private string estado;
```

Se implementó el encapsulamiento en la declaración de los atributos de las clases. Siguiendo las indicaciones establecidas al inicio del curso, todos los atributos fueron declarados con el modificador de acceso private, evitando que puedan ser modificados directamente desde otras clases.

Para controlar el acceso y modificación de estos atributos, se utilizaron métodos `get` y `set`, incorporando las validaciones correspondientes dentro de los `set`.

En los atributos de tipo `int` y `double`, se establecieron condiciones para evitar el ingreso de valores negativos. En los atributos de tipo `string`, se implementó una validación para evitar que sean ingresados valores vacíos.

Estas validaciones permiten mantener la integridad de los datos y evitar que el sistema almacene información que no cumpla con las condiciones establecidas.

Dificultades en la parte de desarrollo

Dificultades durante la fase uno

Durante la fase uno, una de las principales dificultades fue definir la lógica que tendría el programa, especialmente la selección de los tipos de variables y las relaciones que existirían entre las diferentes clases.

Al inicio se planteó establecer una relación directa entre los vehículos y los repartidores, considerando que cada repartidor tendría asociado un vehículo. Sin embargo, al desarrollar la estructura del programa se identificó que esta relación no se adaptaba correctamente a la lógica y funcionamiento del sistema.

Por esta razón, fue necesario replantear la estructura y las relaciones entre las clases, analizando qué información debía pertenecer a cada una y cómo debían interactuar entre sí. Este proceso permitió establecer una estructura más adecuada para el funcionamiento del programa.

Dificultado durante la fase de desarrollo

Inicialmente, se creó un primer repositorio con el objetivo de realizar pruebas relacionadas con el funcionamiento del control de versiones, especialmente en la creación, uso y manejo de ramas. Este repositorio permitió experimentar con la estructura de trabajo antes de implementar el repositorio definitivo del proyecto.

Posteriormente, se creó el segundo repositorio, el cual fue utilizado como repositorio principal del proyecto. Durante el proceso de desarrollo se presentaron algunos conflictos al momento de realizar los `commits` y subir los cambios al repositorio, principalmente debido a modificaciones realizadas en los archivos y al proceso de integración de los cambios.

Estas dificultades permitieron comprender mejor el funcionamiento del control de versiones, el manejo de ramas y la resolución de conflictos durante el desarrollo colaborativo del proyecto.

Menú principal

```
=====
      SISTEMA DE GESTIÓN GOXELA DELIVERY
=====
1. Registrar Vehículo
2. Registrar Paquete
3. Mostrar Flota y Paquetes
4. Reporte Especial
5. Registrar Cliente
6. Registrar Repartidor
7. Crear Entrega
8. Asignar Repartidor y Vehículo a Entrega
9. Cambiar Estado de Entrega
10. Registrar Incidencia
11. Ver Reportes Generales
12. Salir
=====
Seleccione una opción:
```

Menú de registro de vehiculo

```
--- REGISTRO DE VEHICULO ---
1. Bicicleta
2. Motocicleta
3. Automóvil
Seleccione el tipo: |
```

Ingreso de datos y especificación

```
--- REGISTRO DE VEHICULO ---
1. Bicicleta
2. Motocicleta
3. Automóvil
Seleccione el tipo: 1
Ingrese código: 002
Ingrese marca: ciclo
Ingrese modelo: 2020
Ingrese capacidad máxima (kg): 50
Ingrese costo operativo por km: 5
```

Registro de paquete

```
--- REGISTRO DE PAQUETE ---
1. Documento
2. Paquete Estandar
3. Paquete Fragil
4. Producto Refrigerado
Seleccione el tipo: 1
Ingrese codigo: 001
Ingrese descripción: documentacion tecnica
Ingrese peso (kg): 0.5
Ingrese valor declarado (Q): 14
Ingrese dirección de origen: 1324
Ingrese dirección de destino: 2324
Paquete registrado exitosamente!
Presione cualquier tecla para continuar...
```

Lista de vehículos subidos

```
--- LISTADO DE VEHÍCULOS ---
Código: 002 | Tipo: Bicicleta | Capacidad: 50kg
--- LISTADO DE PAQUETES ---
Código: 001 | Descripción: documentacion tecnica
Tarifa estimada (10 km): Q35
Presione cualquier tecla para regresar al menú...
```

Reporte de capacidad

```
--- REPORTE DE CAPACIDAD TOTAL ---  
Cantidad de vehículos: 1  
Capacidad total acumulada: 50 kg  
  
Presione cualquier tecla para regresar al menú...
```

Crear entrega

```
--- CREAR ENTREGA ---  
Ingrese código de la entrega: 009  
Ingrese código del cliente: 007  
Error: no existe un cliente con ese código.
```

Registro de cliente

```
--- REGISTRO DE CLIENTE ---  
Ingrese código: 001  
Ingrese nombre completo: juanito  
Ingrese teléfono: 1234569  
Ingrese dirección: calle 2  
Ingrese correo: algienq@gmail.com  
Cliente registrado exitosamente.  
Presione cualquier tecla para continuar...
```

Registro de paquetes

```
--- REGISTRO DE PAQUETE ---  
1. Documento  
2. Paquete Estandar  
3. Paquete Frágil  
4. Producto Refrigerado  
Seleccione el tipo: 2  
Ingrese código: 09  
Ingrese descripción: un computador  
Ingrese peso (kg): 2  
Ingrese valor declarado (Q): 122  
Ingrese dirección de origen: 21  
Ingrese dirección de destino: 31  
Paquete registrado exitosamente!  
Presione cualquier tecla para continuar...
```

Entrega

--- CREAR ENTREGA ---

Ingrese código de la entrega: 002

Ingrese código del cliente: 001

Ingrese código del paquete: 09

Ingrese dirección de origen: